PROGRAMAÇÃO WEB
PROPOSTA DE ATIVIDADE AVALIATIVA

Trabalho em Grupo 1

Desenvolvimento de uma aplicação Web com consumo de API

INFORMAÇÕES GERAIS

MODALIDADE **Atividade prática em grupo**	FORMAÇÃO DOS GRUPOS **Aproximadamente 1 - 5 integrantes**
VALOR **10,0 pontos**	TECNOLOGIAS **HTML, CSS e JavaScript**
FORMA DE ENTREGA **GitHub e SIGAA**	PRAZO DE ENTREGA **a definir**

1 Tema e proposta

Catálogo interativo de produtos

Desenvolver uma aplicação Web que consulte produtos de uma **API pública**, permita **pesquisar**, exibir **detalhes** e manter uma lista de **favoritos**. Utilize exclusivamente **HTML, CSS e JavaScript** (sem frameworks ou bibliotecas): os dados vêm por requisições HTTP e as interações são feitas via DOM.

O objetivo é integrar os conteúdos da disciplina DOM, eventos, objetos, JSON, `localStorage`, APIs, HTTP e Fetch API. **Não há servidor próprio nem banco de dados**: todo o trabalho é frontend, executado no navegador.

2 Requisitos obrigatórios

2.1. Interface com HTML e CSS

- Interface organizada e responsiva.
- HTML semântico.
- Estilos próprios com CSS.
- Produtos em lista ou cartões.

2.2. JavaScript e manipulação do DOM

- Variáveis, condicionais, funções e laços de repetição.
- Manipular elementos HTML por meio do DOM.
- Criar elementos dinamicamente.
- Implementar eventos de clique e de formulário.

2.3. Objetos e JSON

- Trabalhar com objetos retornados pela API.
- Percorrer arrays de objetos.
- Compreender e utilizar os dados em JSON.
- Criar objetos JavaScript para representar os favoritos.

2.4. APIs e requisições HTTP

- Consumir uma API pública com `fetch()`.
- Utilizar o método GET.
- Implementar requisições assíncronas com `async/await`.
- Tratar respostas HTTP e erros de rede.
- Exibir os resultados dinamicamente na interface.

2.5. Persistência com localStorage

- Permitir que o usuário favorite produtos.
- Armazenar os favoritos no `localStorage`.
- Utilizar `JSON.stringify()` e `JSON.parse()`.
- Recuperar os favoritos ao abrir ou recarregar a página.
- Permitir remover produtos dos favoritos.

3 Funcionalidades mínimas

Funcionalidade	Descrição
Listagem	Exibir produtos da API com nome, imagem e preço.
Pesquisa	Pesquisar produtos por nome.
Detalhes	Mostrar informações adicionais de um produto selecionado.
Favoritos	Adicionar e remover produtos da lista de favoritos.
Persistência	Manter os favoritos após recarregar a página.
Carregamento	Exibir uma mensagem enquanto os dados carregam.
Tratamento de erros	Avisar o usuário quando a API falhar ou estiver indisponível.

4 APIs públicas sugeridas

Recomendada: DummyJSON — Products API

Dados fictícios de produtos, **sem chave de autenticação**. Suporta busca por `?q=`.

Documentação: https://dummyjson.com/docs/products `https://dummyjson.com/products`

Alternativas públicas e gratuitas (sem chave) — todas permitem listar, pesquisar, ver detalhes e favoritar:

API	Endpoint inicial	Observação
Fake Store API	`https://fakestoreapi.com/products`	Produtos com título, preço, imagem e categoria.
Platzi Fake Store	`https://api.escuelajs.co/api/v1/products`	Produtos com imagens, categorias e paginação.
TheMealDB	`https://www.themealdb.com/api/json/v1/1/search.php?s=`	Catálogo de receitas (tema alternativo).
PokéAPI	`https://pokeapi.co/api/v2/pokemon`	Catálogo de Pokémon (tema alternativo).
REST Countries	`https://restcountries.com/v3.1/all`	Catálogo de países (tema alternativo).

Outra API pública poderá ser usada, desde que seja adequada à proposta e permita implementar as funcionalidades exigidas.

5 Critérios de avaliação

Critério	Descrição	Pontos
HTML e CSS	Organização, semântica, apresentação e responsividade.	**1,5**
JavaScript e DOM	Funções, eventos, formulários e atualização dinâmica da interface.	**2,0**
Consumo da API e HTTP	Fetch, async/await, interpretação de respostas e tratamento de erros.	**2,5**
Objetos, JSON e localStorage	Manipulação de dados, favoritos e persistência.	**2,0**
Organização e funcionamento	Organização dos arquivos, testes e funcionamento geral.	**2,0**
Total		**10,0**

A avaliação considera os arquivos entregues e o funcionamento da aplicação; **não haverá apresentação oral**. Requisitos não implementados, funcionalidades incompletas e erros de execução afetam a pontuação do respectivo critério.

6 Entrega

Dois canais obrigatórios: GitHub e SIGAA

- **GitHub:** repositório com todo o código e um `README.md` (nome do projeto, integrantes, funcionalidades e como executar). Público ou privado, desde que o professor tenha acesso.
- **SIGAA:** arquivo `.zip` com o código completo (HTML, CSS, JS, imagens e `README.md`), nomeado como `Trabalho_Grupo_01.zip` (o número corresponde ao grupo).
- Informe o **link do repositório** no campo de observações do SIGAA.
- Os dois envios são obrigatórios, um não substitui o outro. Antes de enviar, confira se os arquivos estão completos e se a aplicação funciona.

O objetivo é aplicar, de forma integrada, os conhecimentos das aulas de Programação Web, construindo uma aplicação funcional que demonstre o domínio do desenvolvimento frontend e do consumo de APIs.